



BudgetBasics

Learn • Plan • Save

PROJECT DOCUMENTATION

Project Name: BudgetBasics
Theme: NextGen BudgetBsic
Category: Web Innovation Unleashed
Institute: Aptech Site Center
Version: 1.0

TEAM WARRIOR CODERS

Team Member	Student ID
TEHZEEB FATIMA	Student1725549
ABDULLAH NAWAZ	Student1727610
MUHAMMAD MAAZ	Student1717941
AHMED RAZA	Student1725525
HIRA NISAR	Student1725855
MUHAMMAD MUSTAFA	Student1725844

A responsive, single-page educational website that teaches students personal budgeting, saving habits, and financial awareness.

Table of Contents

1. Introduction	3
2. Scope and Constraints	4
3. Functional Requirements Implemented	5
4. Non-Functional Requirements	6
5. System Design	7
6. UI / Visual Design	9
7. Technology Stack	10
8. Project Folder Structure	11
9. Test Data Used	12
10. Installation Instructions	13
11. Assumptions	14
12. Screenshots	15
13. Source Code with Comments	34
14. AI Tool Acknowledgement	42
15. Conclusion	43

1. Introduction

1.1 Background and Necessity

Managing personal finances is an essential life skill. Many students begin handling allowances, scholarships, or part-time income without a simple way to plan and track their money. Small, everyday expenses can quickly erode the funds available for essential needs, learning resources, transport, and savings.

BudgetBasics was built to close this gap — a student-friendly website that turns budgeting concepts into clear explanations, relatable examples, visual infographics, and interactive tools, helping learners understand where their money comes from, where it goes, and how small planning decisions build better habits.

1.2 Problem Definition

Students frequently lack an accessible, engaging way to learn budgeting fundamentals. Existing personal-finance tools are either too complex, aimed at adults with bank accounts, or not designed with an educational, beginner-friendly lens. There is a need for a lightweight, front-end-only, visually engaging website that teaches — rather than manages — personal finance.

1.3 Proposed Solution

BudgetBasics is a responsive, single-page educational website built with HTML5, CSS3, and vanilla JavaScript (no backend, no database, no login). It teaches budgeting fundamentals through guided sections, a Needs vs. Wants classifier, a live 50-30-20 budget calculator, a savings-goal planner, an expense planner, a money-mistakes gallery, infographics, a searchable topic index, and a rule-based BudgetBasics chatbot — all styled around one consistent teal-and-gold visual identity.

1.4 Objectives

- Provide a simple, student-friendly platform for understanding budgeting concepts.
- Help users classify income into needs, wants, and savings using the 50-30-20 rule.
- Demonstrate common money mistakes with realistic scenarios and corrective actions.
- Offer interactive calculators for budgeting and savings-goal planning.
- Deliver a searchable, filterable library of budgeting topics.
- Provide instant, predefined-answer support through a floating chatbot.
- Ensure full responsiveness across desktop, tablet, and mobile devices.
- Keep all content data-driven via external JSON files rather than hardcoded markup.

2. Scope and Constraints

2.1 Scope

BudgetBasics is a purely front-end, single-page educational website. It demonstrates budgeting concepts and calculations for learning purposes only, and does not process real transactions, store banking details, or connect to any financial institution.

In Scope	Out of Scope
Budgeting education & infographics	Real banking integration
Client-side calculators (50-30-20, savings, expenses)	Persistent server-side storage
Rule-based educational chatbot	AI/LLM-powered chatbot
Responsive design (desktop / tablet / mobile)	User accounts / authentication
Light & dark theme	Real transaction processing

2.2 Constraints

- No backend services, server-side database, or login/session storage.
- No real financial transactions are processed or stored.
- All demonstration data is temporary and session-based only (cleared on refresh).
- Content is retrieved from static HTML, JavaScript, and pre-populated JSON files.

3. Functional Requirements Implemented

The table below maps every SRS-mandated module to its implementation status on the BudgetBasics website.

#	Module	Key Features Implemented
1	Navigation & Hero	Sticky header, responsive nav, hero banner with live 50-30-20 budget preview card.
2	Budgeting Basics	Income, fixed/variable expenses, savings cards; sample monthly budget table; knowledge-check quiz.
3	Needs vs. Wants	Side-by-side classification cards with color-coded (teal / gold) icon lists.
4	50-30-20 Rule	Explanation cards + live income calculator with instant needs/wants/savings breakdown.
5	Savings Goals	Goal name, target, current savings & monthly contribution → remaining amount + estimated months.
6	Expense Planner	Add / edit / delete session expenses, category badges, running total, budget progress bar.
7	Money Mistakes	5 common mistake cards with icon, scenario, and corrective guidance.
8	Infographics	Visual donut-style charts explaining the 50-30-20 split.
9	Search & Filter	Live keyword search across budgeting topic cards.
10	BudgetBsic Assistant	Rule-based chatbot (inline section + floating widget) answering predefined budgeting questions from JSON.
11	About / Feedback	Project info card + client-side validated feedback form with rating.
12	Sitemap & Footer	Full site navigation, social links, footer branding.
13	Dark Mode	Toggle with persisted color tokens across every component.
14	Scroll Animations	AOS-powered zoom-in entrance animations, directionally mapped to layout.

4. Non-Functional Requirements

Requirement	How BudgetBasics Meets It
Safe to Use	No downloads triggered; all external links open clearly in a new context.
Accessibility	Semantic HTML, labeled form fields, visible focus states, keyboard-navigable menu and buttons.
User-Friendliness	Plain language, consistent card-based layout, clear success/error feedback on every form.
Operability	All calculators, forms, filters, and the chatbot function reliably with valid input.
Performance	Lightweight vanilla JS, no heavy frameworks, smooth AOS-based transitions.
Reliability	Input validation prevents blank/negative/non-numeric values from breaking calculations.
Scalability	Content (topics, quiz, mistakes, chatbot answers) is data-driven via JSON — new items can be added without touching markup.
Compatibility	Tested responsive layout across Chrome, Edge, and mobile viewport widths.
Maintainability	Organized folder structure (/css, /js, /data, /image) with commented, modular script.js.
Privacy	No sensitive or banking data is collected; all entries are session-only and never transmitted.
Accuracy	50-30-20 and savings-goal formulas are fixed, labeled, and explicitly presented as educational estimates.

5. System Design

5.1 Information Architecture (Sitemap)

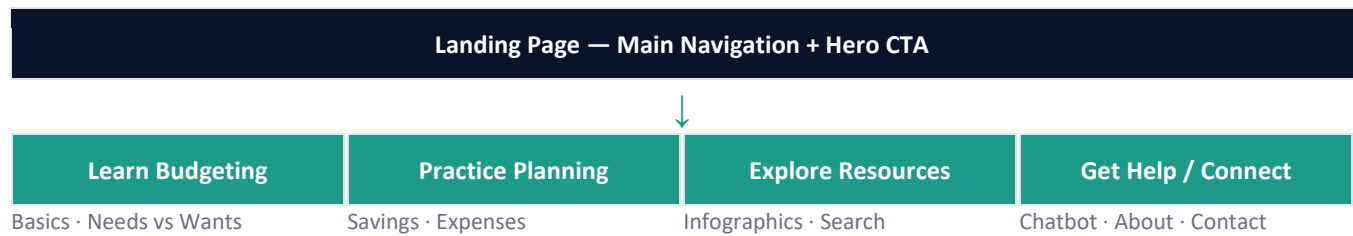


Fig 5.1 — BudgetBasics information architecture

5.2 Application Flow (High-Level)

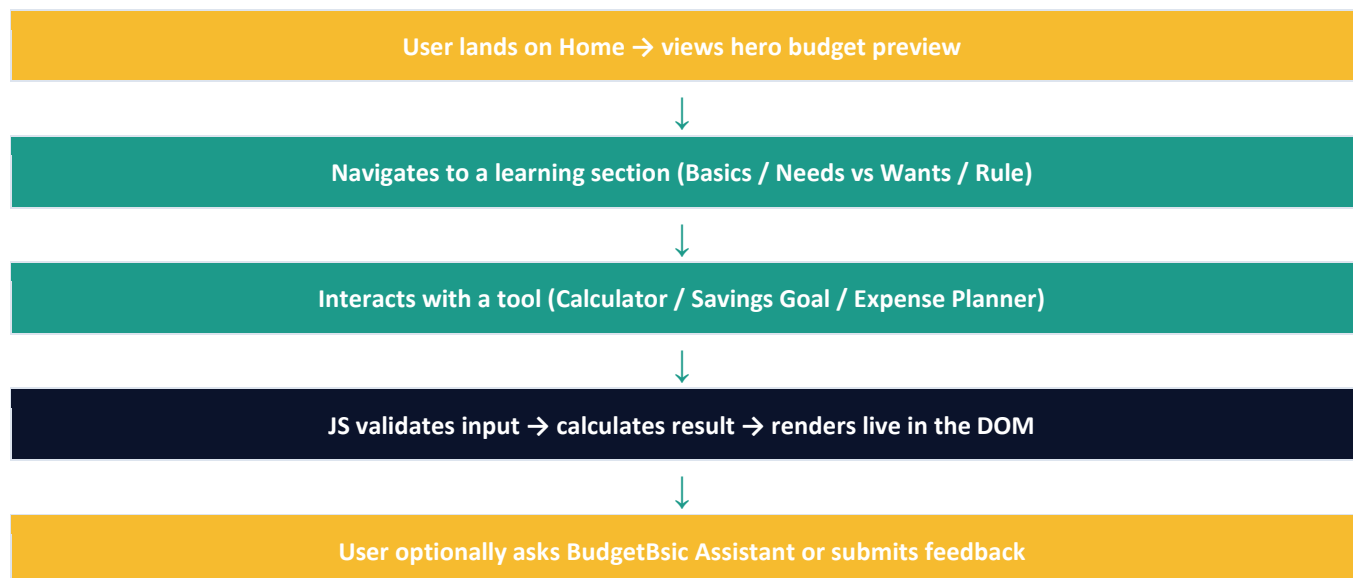


Fig 5.2 — End-to-end user interaction flow

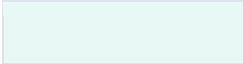
5.3 Data Flow (Level 0)

Source	Process	Destination
User input (income, goal, expense)	JavaScript validation + calculation	DOM update (result cards, progress bars)
/data/*.json (topics, quiz, mistakes, chatbot, categories)	fetch() on page load	Dynamically rendered cards / lists
User keyword input	Chatbot keyword-matching logic	Predefined JSON answer displayed
Feedback / contact form	Client-side validation only	On-screen confirmation message (not transmitted)

6. UI / Visual Design

6.1 Color Palette

The interface uses a consistent teal-and-gold identity across light and dark themes, defined as CSS custom properties for easy maintenance.

Swatch	Token	Hex	Usage
	Primary	#1D9A8A	Buttons, links, icons, strips
	Primary Dark	#117568	Hover states, headings
	Deep Navy	#0B132B	Footer, chatbot background, dark accents
	Accent Gold	#F6BB2F	Highlighted / center cards, CTAs
	Mint	#E8F8F5	Soft card backgrounds, tag chips
	Text	#14213D	Body copy
	Muted	#687386	Secondary text

6.2 Typography & Components

- Font family: Inter / Segoe UI / Arial fallback stack, for clean legibility across devices.
- Cards: rounded corners (22–24px radius), soft shadow, gradient top-strip accent, scale + shadow hover transitions.
- Buttons: gradient fill with icon + label, shine-sweep hover effect, consistent 14px border radius.
- Icons: Font Awesome solid icon set, colored to match each card's accent (primary or gold).
- Motion: AOS (Animate On Scroll) — zoom-in-left / zoom-in-right for two-column sections, zoom-in for headings and card grids.

6.3 Responsiveness

Layouts use CSS Grid and Flexbox with breakpoints at 1100px, 900px, 700px, and 480px, collapsing multi-column card grids down to two, then one column, and converting the navigation into a slide-down mobile menu below 900px.

7. Technology Stack

Layer	Technology
Structure	HTML5 (semantic sections, ARIA-friendly labels)
Styling	CSS3 — custom properties, Grid, Flexbox, media queries
Interactivity	Vanilla JavaScript (ES6+) — no frameworks
Data Store	JSON files (topics, quiz, mistakes, chatbot, categories, saving tips)
Icons	Font Awesome 6
Animation	AOS (Animate On Scroll) library
Testing / Auditing	Google Lighthouse (performance, accessibility, SEO)
Hosting	GitHub Pages (static hosting, HTTPS by default)

8. Project Folder Structure

```
budgetbasics/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── script.js  
├── data/  
│   ├── topics.json  
│   ├── quiz.json  
│   ├── mistakes.json  
│   ├── expenseCategories.json  
│   ├── savingTips.json  
│   ├── chatbot.json  
│   └── sitemap.json  
└── image/  
    └── logo.png
```

9. Test Data Used

The following sample data was used to validate calculators, forms, and dynamically loaded sections.

9.1 Budget Calculator (50-30-20)

Monthly Income (Rs.)	Needs (50%)	Wants (30%)	Savings (20%)
20,000	10,000	6,000	4,000
35,000	17,500	10,500	7,000
0 / blank	Validation error shown	—	—
-500 (negative)	Validation error shown	—	—

9.2 Savings Goal Planner

Goal	Target	Current Savings	Monthly Contribution	Result
New Laptop	60,000	10,000	5,000	10 months remaining
Emergency Fund	30,000	0	0	Validation error (contribution required)

9.3 Sample chatbot.json Entry

```
{
  "keywords": ["50-30-20", "rule", "split"],
  "answer": "The 50-30-20 rule suggests spending 50% of your income on needs, 30% on wants, and saving 20% for future goals."
}
```

10. Installation Instructions

BudgetBasics is a static front-end project and requires no build tools, package installation, or server-side setup for basic viewing. A lightweight local server is only required to enable JSON `fetch()` calls (browsers block `fetch()` on the `file://` protocol).

10.1 Option A — VS Code Live Server (Recommended)

- 1. Install Visual Studio Code (<https://code.visualstudio.com>).
- 2. Install the “Live Server” extension by Ritwick Dey from the Extensions panel.
- 3. Extract the submitted project ZIP file and open the folder in VS Code.
- 4. Right-click `index.html` → “Open with Live Server”.
- 5. The site opens automatically at `http://127.0.0.1:5500` with full JSON functionality.

10.2 Option B — Python Local Server

- 1. Ensure Python 3 is installed (`python --version`).
- 2. Open a terminal inside the project folder.
- 3. Run: `python -m http.server`
- 4. Open `http://localhost:8000` in a browser.

10.3 Browser Requirements

Latest versions of Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari. JavaScript must be enabled.

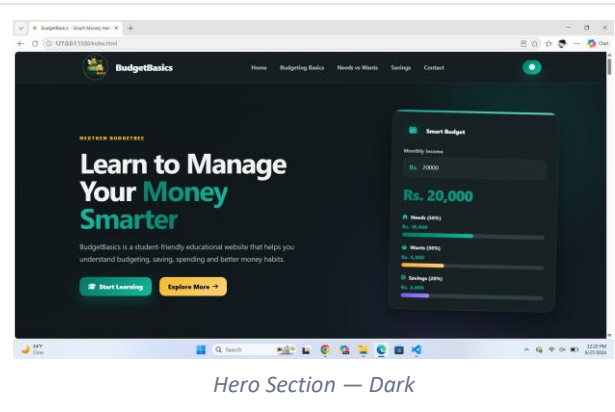
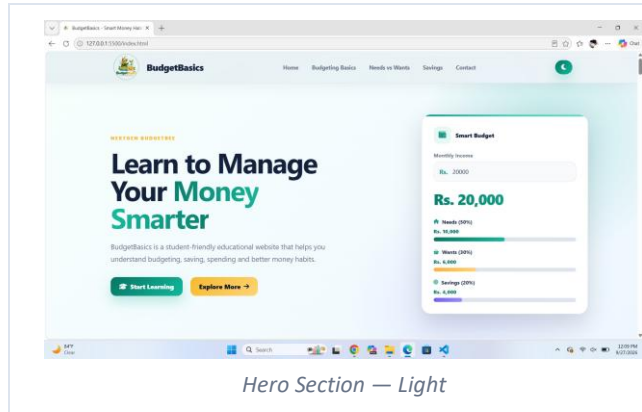
11. Assumptions

- All monetary figures are illustrative sample values for educational purposes only, in Pakistani Rupees (Rs.).
- The website is a single-user, client-side demonstration — no data persists between sessions or browser refreshes.
- The BudgetBsic Assistant answers only from a fixed, predefined JSON knowledge base and does not use any external AI/LLM service.
- Feedback and contact forms perform client-side validation only and do not transmit or store submitted data.
- The site is optimized for modern evergreen browsers; legacy browser support (e.g. Internet Explorer) is out of scope.

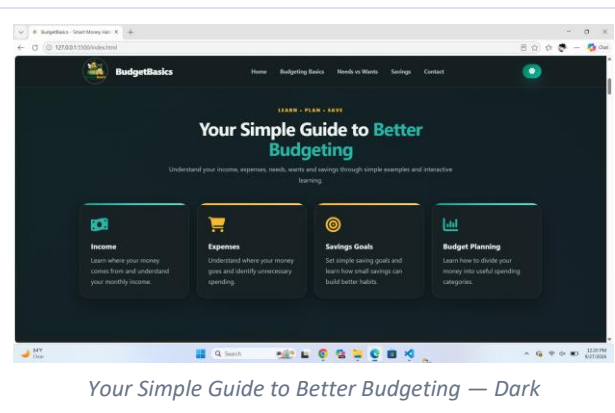
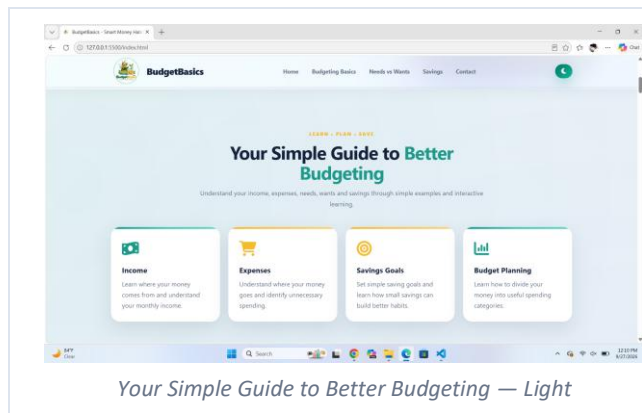
12. Screenshots

The following screenshots capture every major section of BudgetBasics in both light and dark theme, as viewed in a desktop browser.

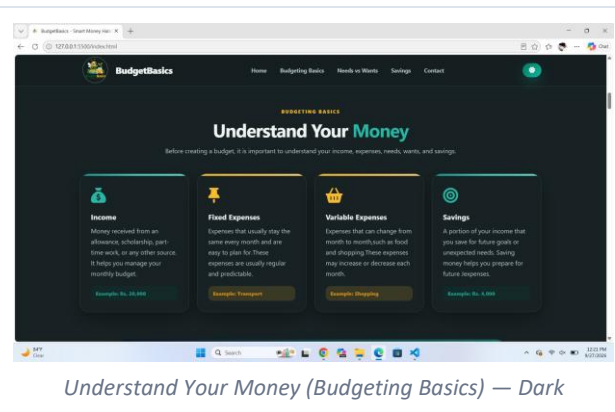
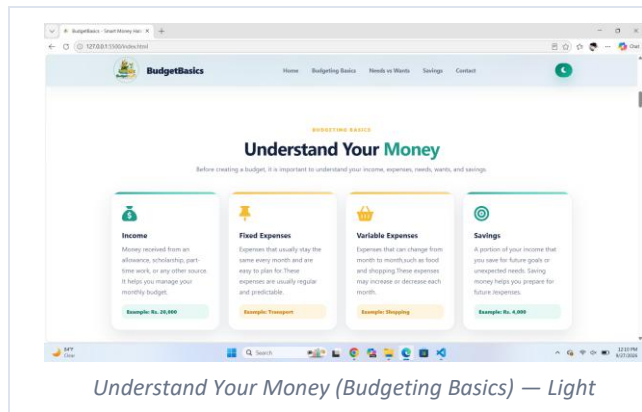
Hero Section



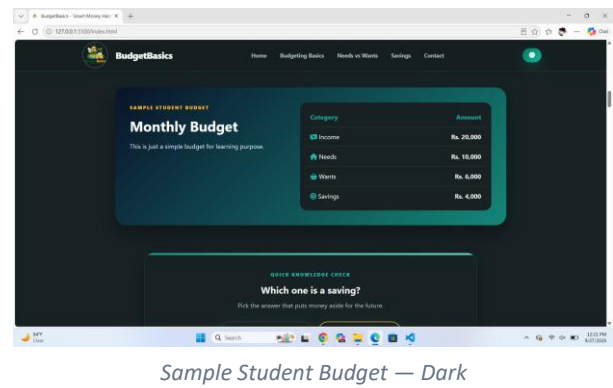
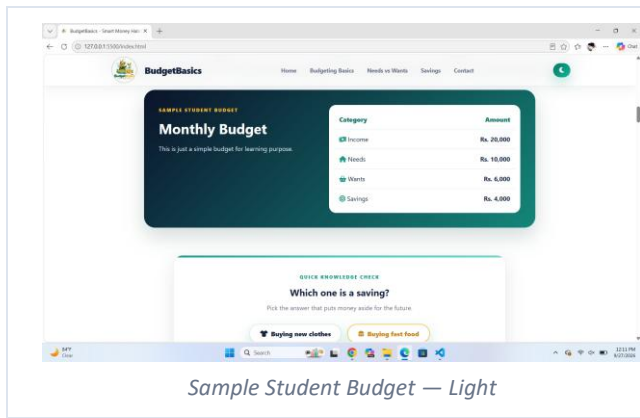
Your Simple Guide to Better Budgeting



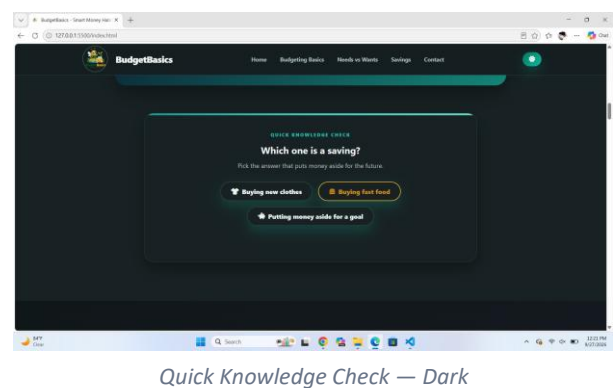
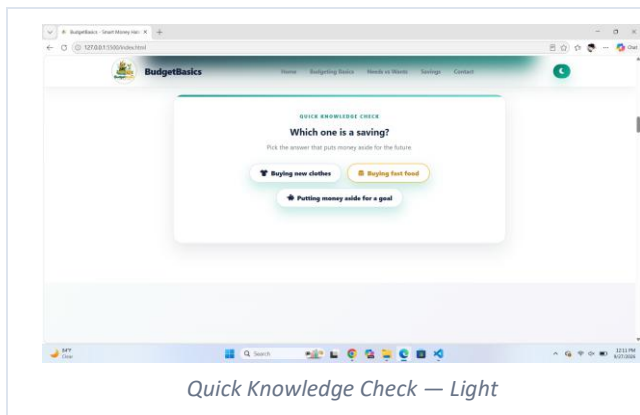
Understand Your Money (Budgeting Basics)



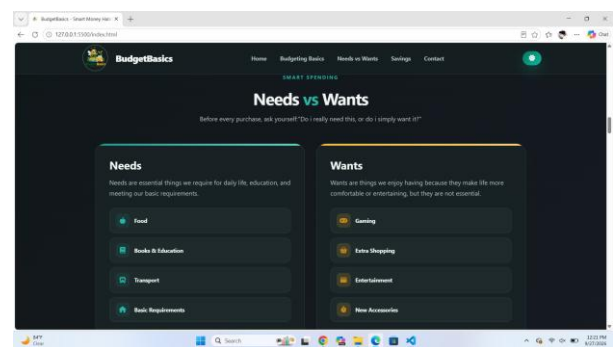
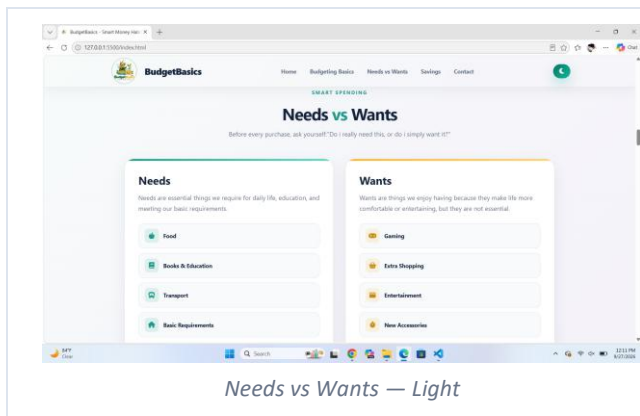
Sample Student Budget



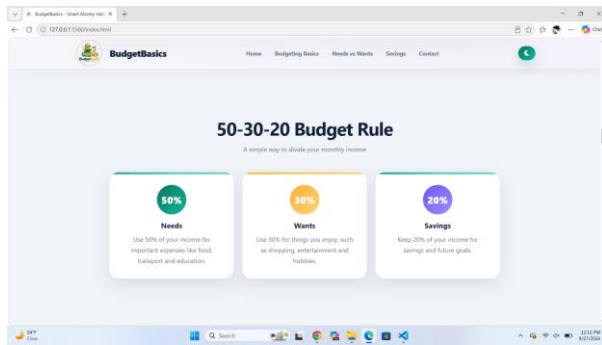
Quick Knowledge Check



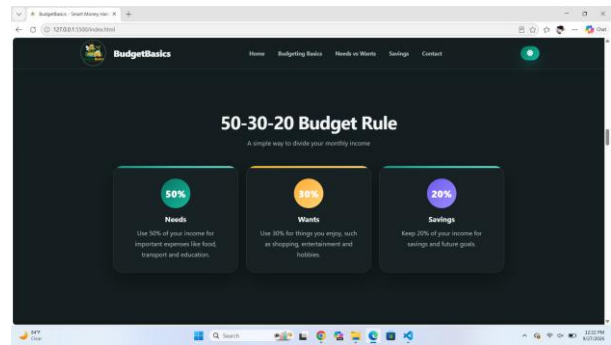
Needs vs Wants



50-30-20 Budget Rule

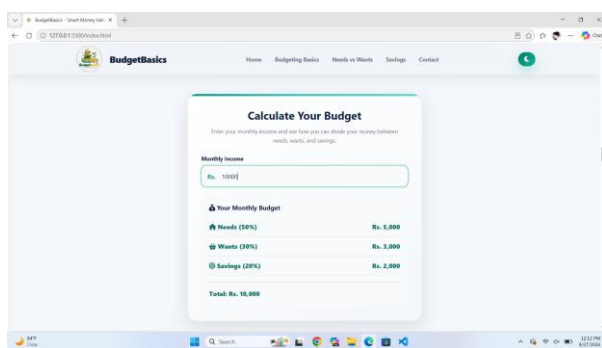


50-30-20 Budget Rule — Light

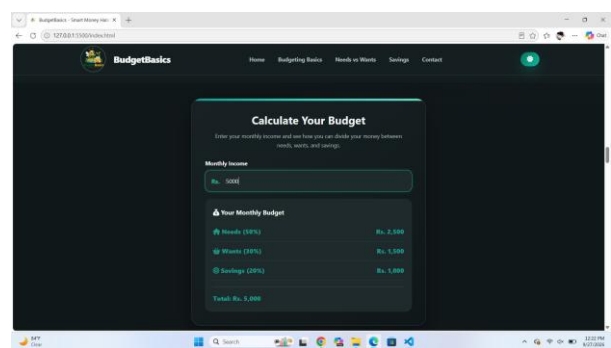


50-30-20 Budget Rule — Dark

Calculate Your Budget

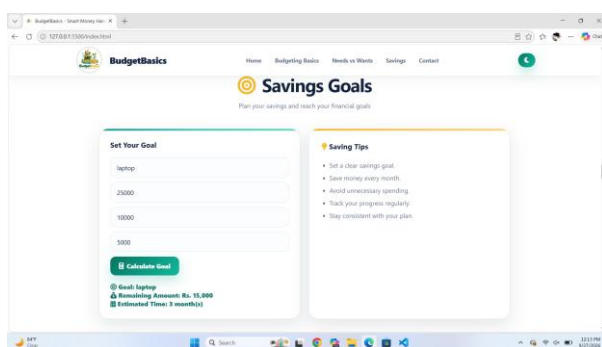


Calculate Your Budget — Light

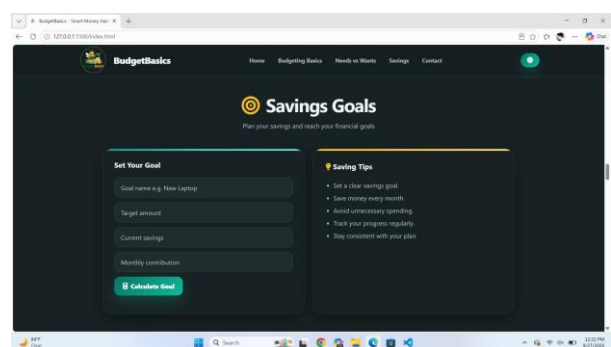


Calculate Your Budget — Dark

Savings Goals

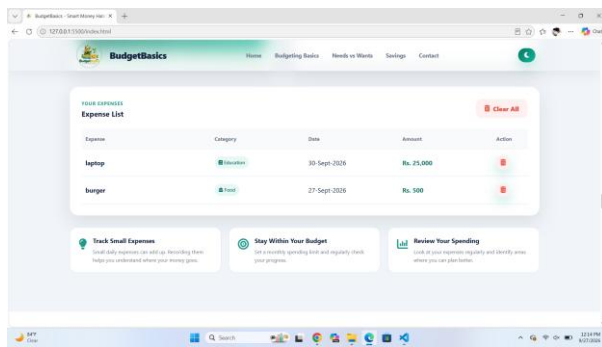


Savings Goals — Light

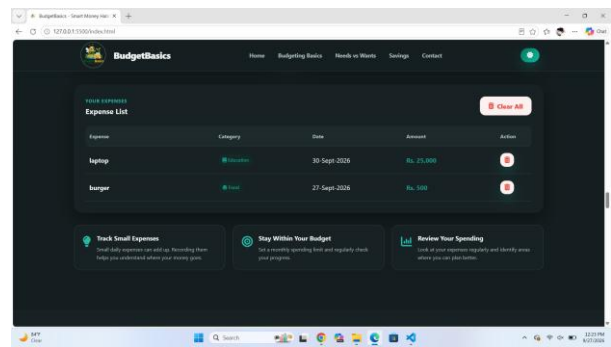


Savings Goals — Dark

Expense Planner — List



Expense Planner — List — Light

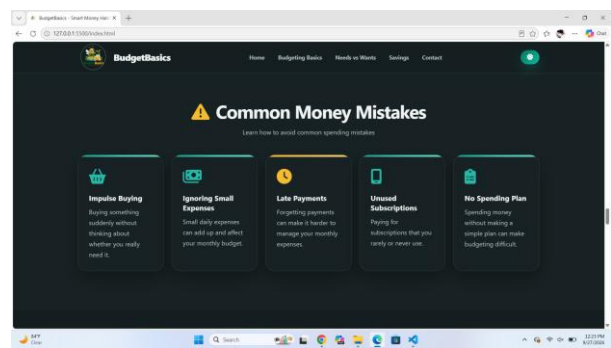


Expense Planner — List — Dark

Common Money Mistakes

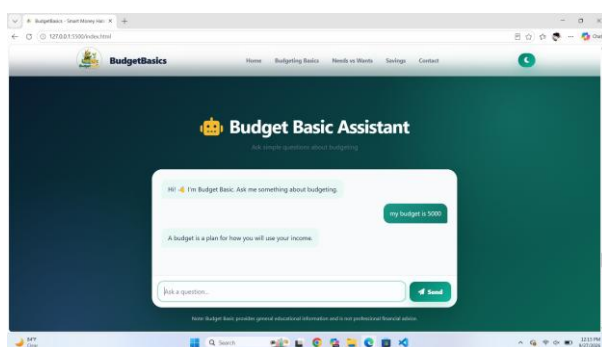


Common Money Mistakes — Light

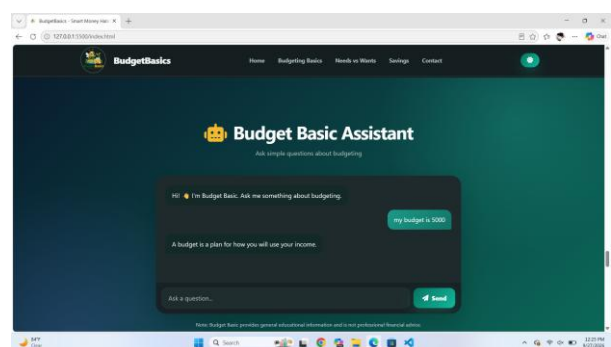


Common Money Mistakes — Dark

Budget Basic Assistant (Chatbot)

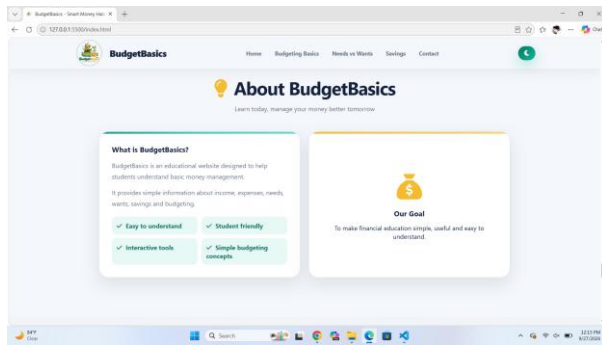


Budget Basic Assistant (Chatbot) — Light

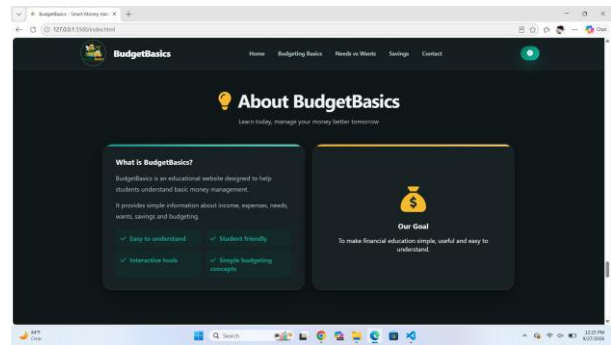


Budget Basic Assistant (Chatbot) — Dark

About BudgetBasics

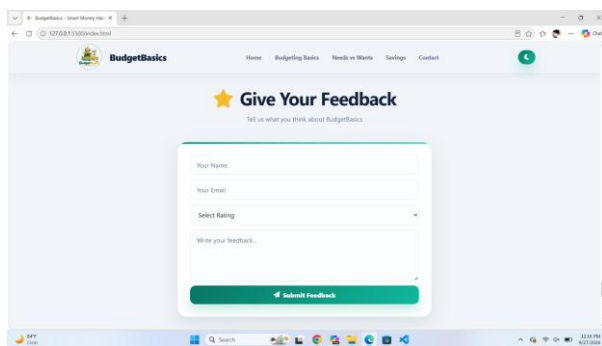


About BudgetBasics — Light

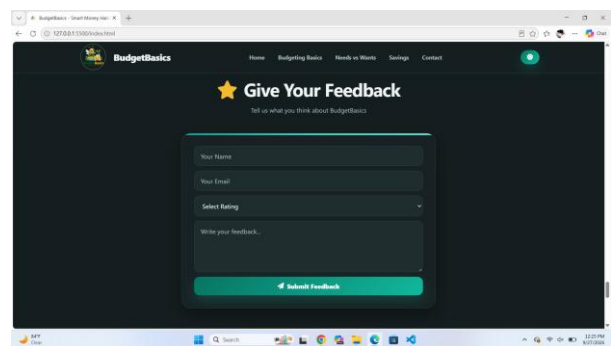


About BudgetBasics — Dark

Give Your Feedback

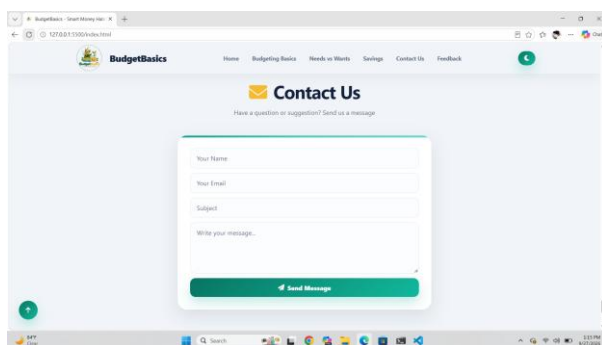


Give Your Feedback — Light

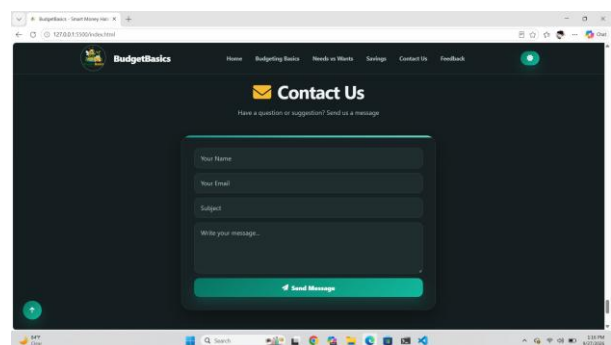


Give Your Feedback — Dark

Contact Us

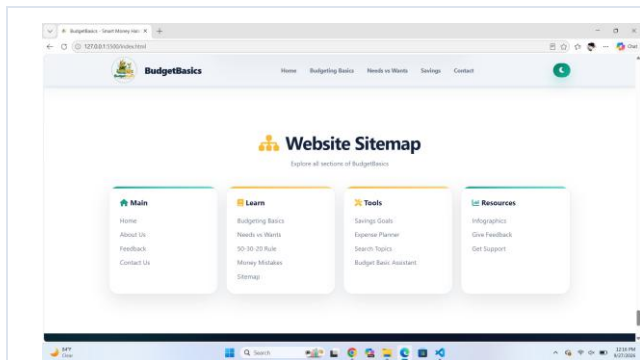


Contact Us — Light

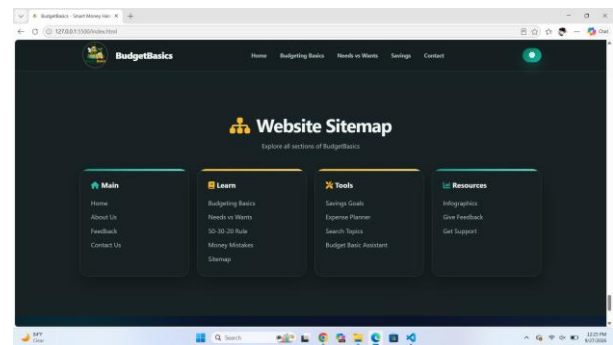


Contact Us — Dark

Website Sitemap



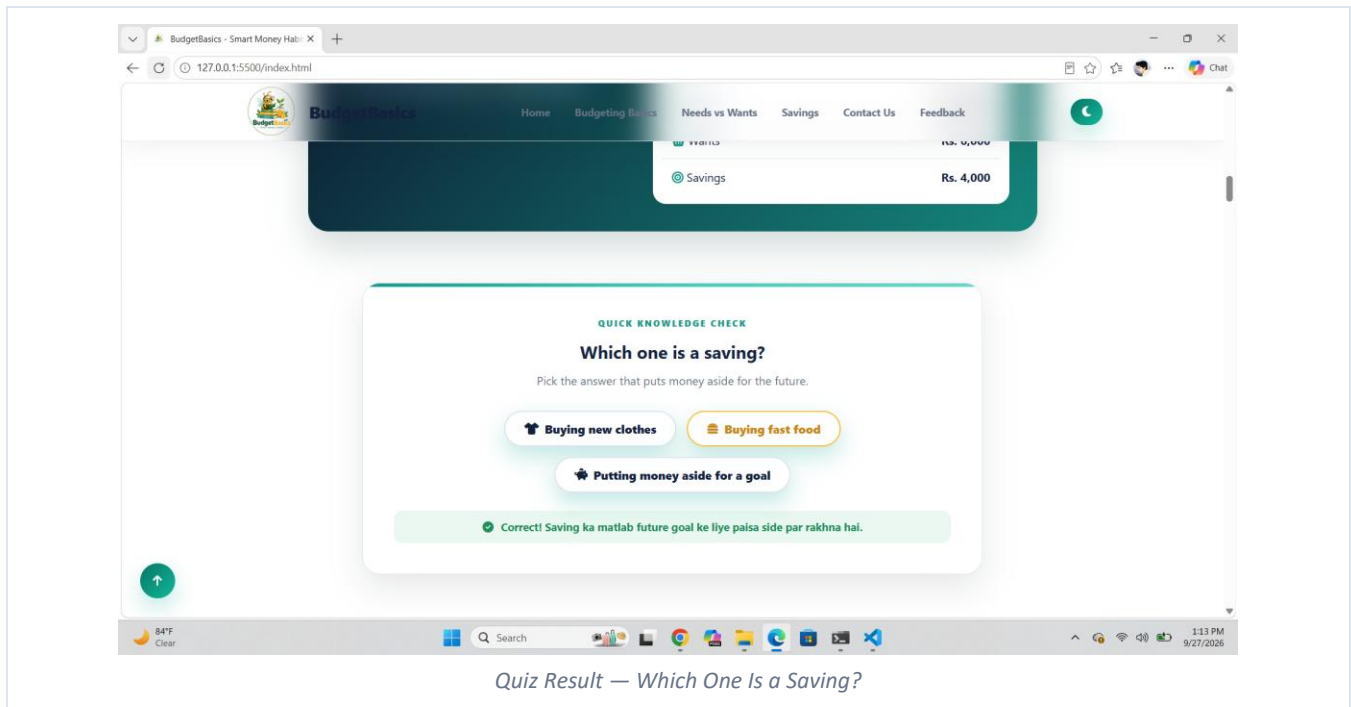
Website Sitemap — Light



Website Sitemap — Dark

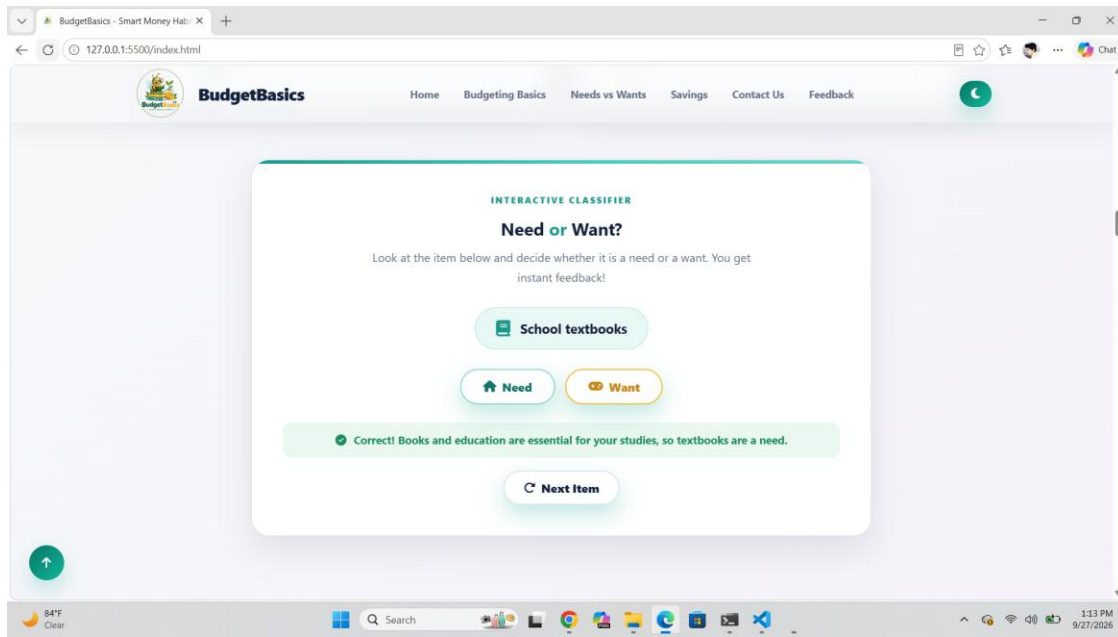
Additional Section States

Quiz Result — Which One Is a Saving?

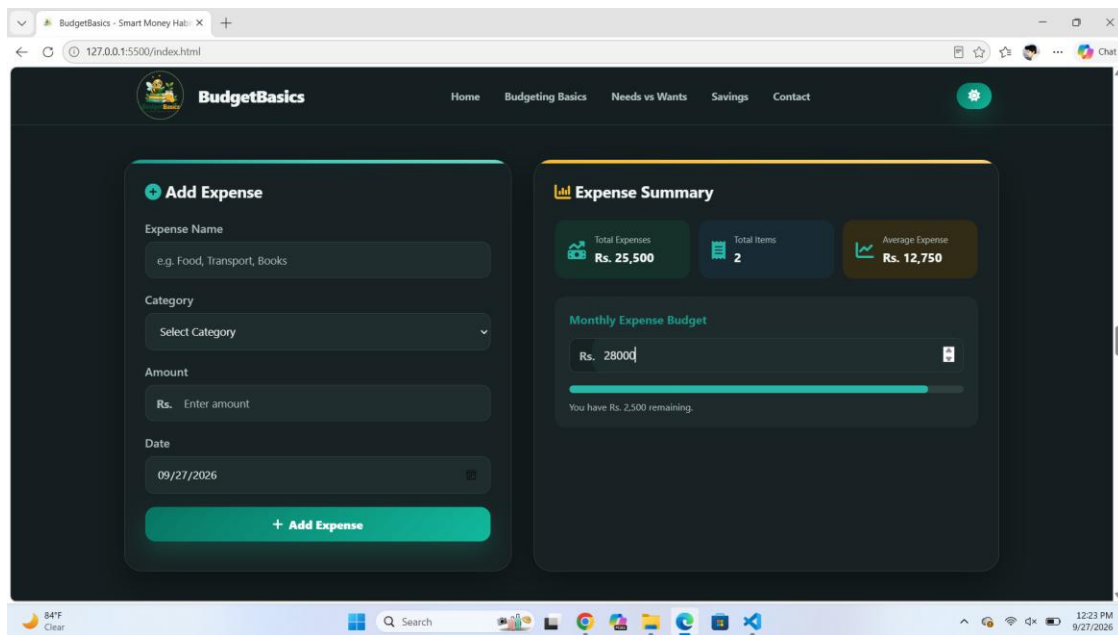


Quiz Result — Which One Is a Saving?

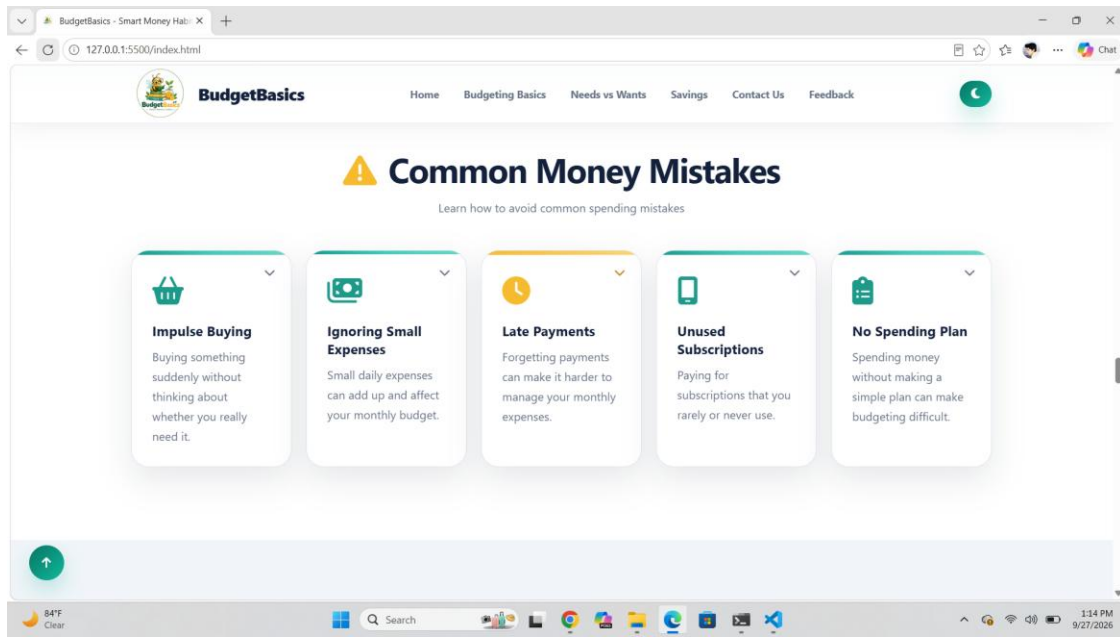
Need or Want — Interactive Prompt

*Need or Want — Interactive Prompt*

Add Expense (Dark Mode)

*Add Expense (Dark Mode)*

Common Money Mistakes — Detail View



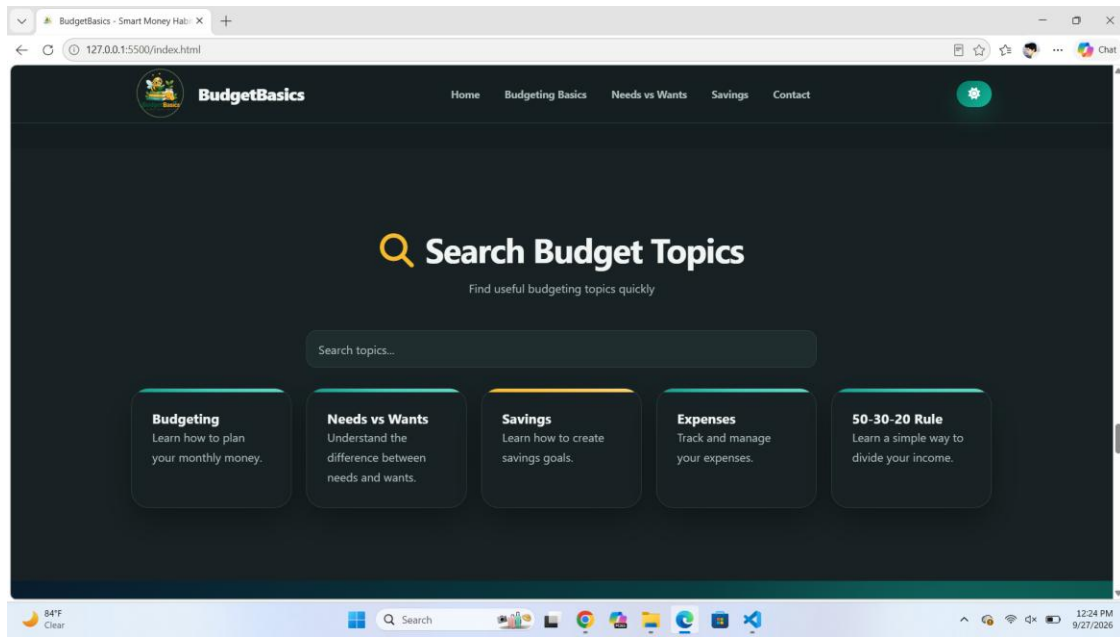
Common Money Mistakes — Detail View

Budget Infographics (Dark Mode)



Budget Infographics (Dark Mode)

Search Budget Topics (Dark Mode)

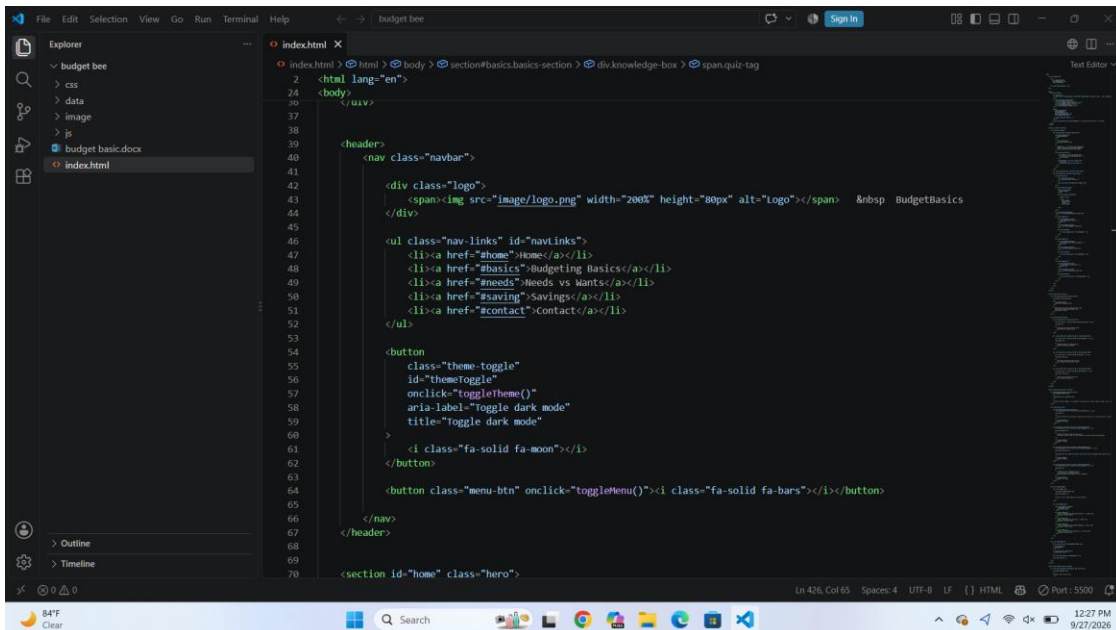


Search Budget Topics (Dark Mode)

13. Source Code with Comments

Representative, commented excerpts from the project's HTML, CSS, and JavaScript source files, illustrating the structure and logic behind key features.

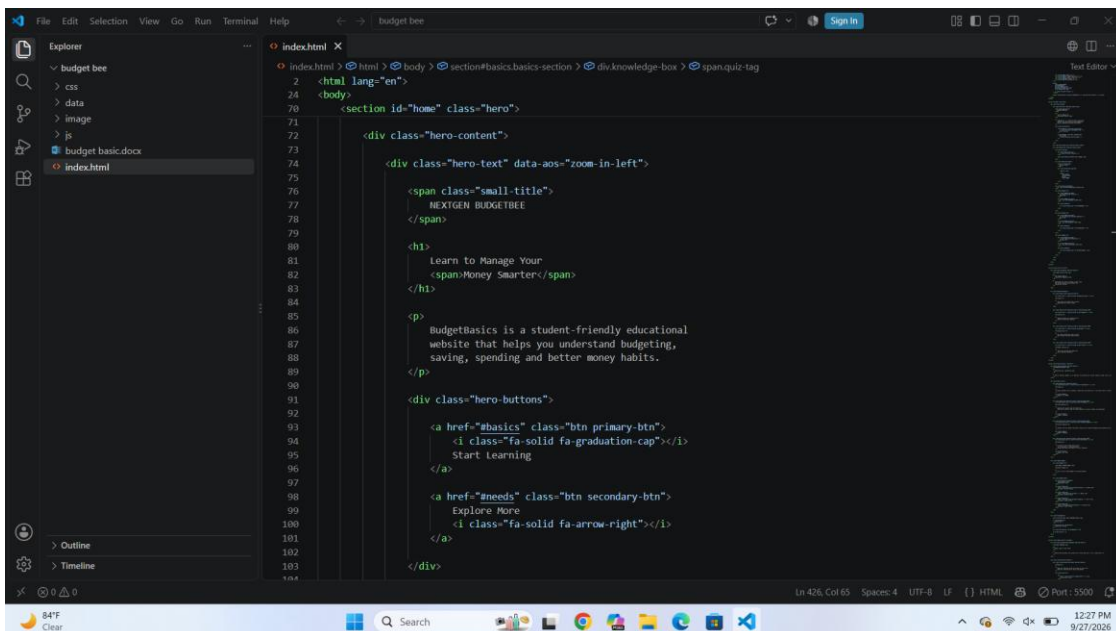
Code Reference 1



```
2 <html lang="en">
24 <body>
37
38
39
40
41 <header>
42   <nav class="navbar">
43     <div class="logo">
44       <span></span> &nbsp; BudgetBasics
45     </div>
46
47     <ul class="nav-links" id="navlinks">
48       <li><a href="#home">Home</a></li>
49       <li><a href="#basics">Budgeting Basics</a></li>
50       <li><a href="#needs">Needs vs Wants</a></li>
51       <li><a href="#savings">Savings</a></li>
52       <li><a href="#contact">Contact</a></li>
53     </ul>
54
55     <button
56       class="theme-toggle"
57       id="themeToggle"
58       onclick="toggleTheme()"
59       aria-label="Toggle dark mode"
60       title="Toggle dark mode"
61     >
62       <i class="fa-solid fa-moon"></i>
63     </button>
64
65     <button class="menu-btn" onclick="toggleMenu()"><i class="fa-solid fa-bars"></i></button>
66   </nav>
67 </header>
68
69 <section id="home" class="hero">
```

Code Reference 1

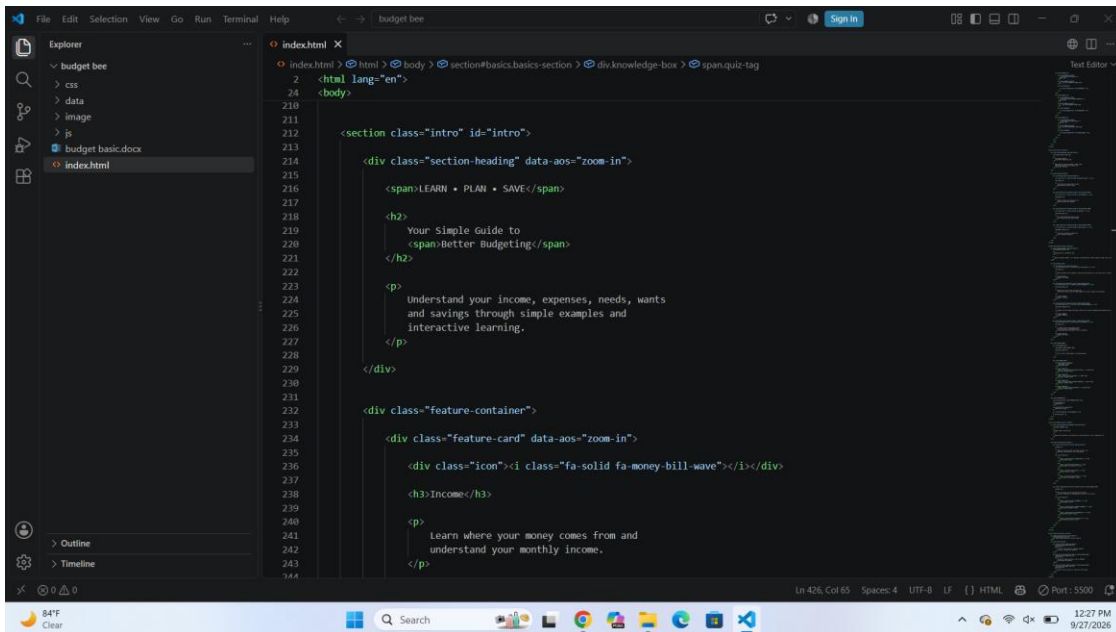
Code Reference 2



```
70 <section id="home" class="hero">
71
72   <div class="hero-content">
73
74     <div class="hero-text" data-aos="zoom-in-left">
75
76       <span class="small-title">
77         NEXTGEN BUDGETBEE
78       </span>
79
80       <h1>
81         Learn to Manage Your
82         <span>Money Smarter</span>
83       </h1>
84
85       <p>
86         BudgetBasics is a student-friendly educational
87         website that helps you understand budgeting,
88         saving, spending and better money habits.
89       </p>
90
91       <div class="hero-buttons">
92
93         <a href="#basics" class="btn primary-btn">
94           <i class="fa-solid fa-graduation-cap"></i>
95           Start Learning
96         </a>
97
98         <a href="#needs" class="btn secondary-btn">
99           Explore More
100          <i class="fa-solid fa-arrow-right"></i>
101        </a>
102      </div>
103    </div>
```

Code Reference 2

Code Reference 3



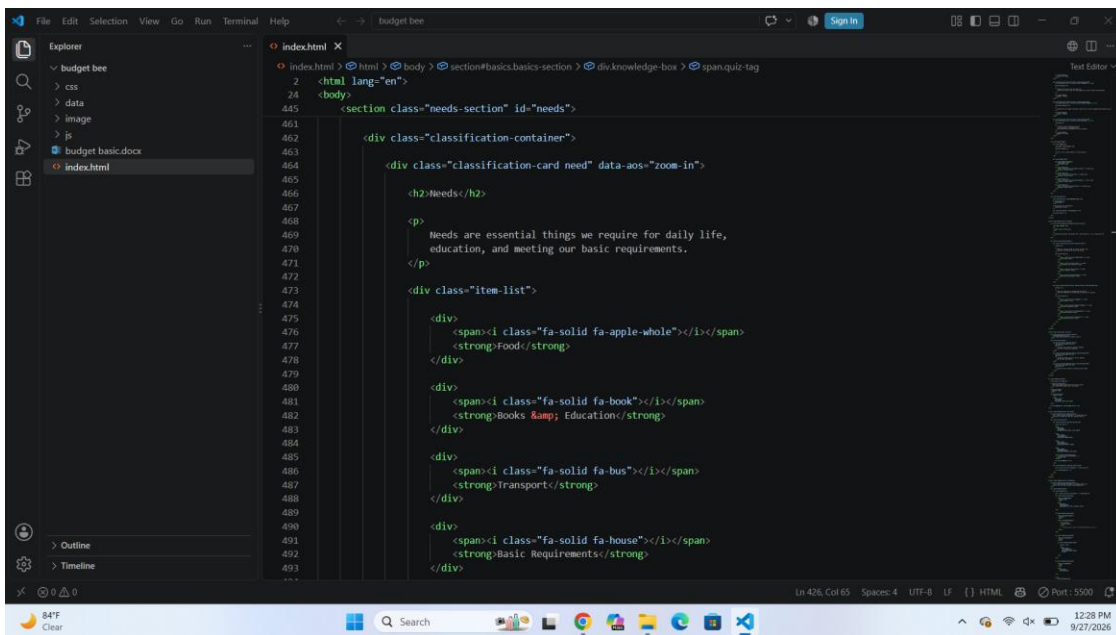
```

2  <html lang="en">
24 <body>
210
211
212
213 <section class="intro" id="intro">
214
215 <div class="section-heading" data-aos="zoom-in">
216
217 <span>LEARN • PLAN • SAVE</span>
218
219 <h2>
220 <span>Your Simple Guide to
221 <span>Better Budgeting</span></h2>
222
223 <p>
224 Understand your income, expenses, needs, wants
225 and savings through simple examples and
226 interactive learning.
227 </p>
228 </div>
229
230
231 <div class="feature-container">
232
233 <div class="feature-card" data-aos="zoom-in">
234
235 <div class="icon"><i class="fa-solid fa-money-bill-wave"></i></div>
236
237 <h3>Income</h3>
238
239 <p>
240 Learn where your money comes from and
241 understand your monthly income.
242 </p>
243
244

```

Code Reference 3

Code Reference 4



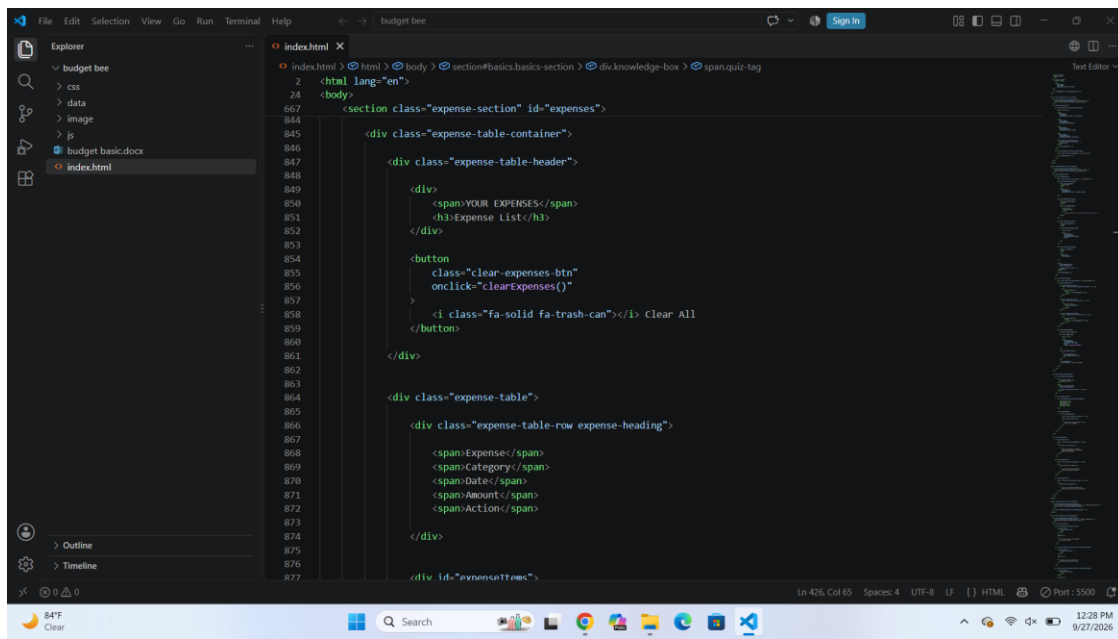
```

461
462
463 <section class="needs-section" id="needs">
464
465 <div class="classification-container">
466
467 <div class="classification-card need" data-aos="zoom-in">
468
469 <h2>Needs</h2>
470
471 <p>
472 Needs are essential things we require for daily life,
473 education, and meeting our basic requirements.
474 </p>
475
476 <div class="item-list">
477
478 <div>
479 <span><i class="fa-solid fa-apple-whole"></i></span>
480 <strong>Food</strong>
481 </div>
482
483 <div>
484 <span><i class="fa-solid fa-book"></i></span>
485 <strong>Books & Education</strong>
486 </div>
487
488 <div>
489 <span><i class="fa-solid fa-bus"></i></span>
490 <strong>Transport</strong>
491 </div>
492
493 <div>
494 <span><i class="fa-solid fa-house"></i></span>
495 <strong>Basic Requirements</strong>
496 </div>
497

```

Code Reference 4

Code Reference 5



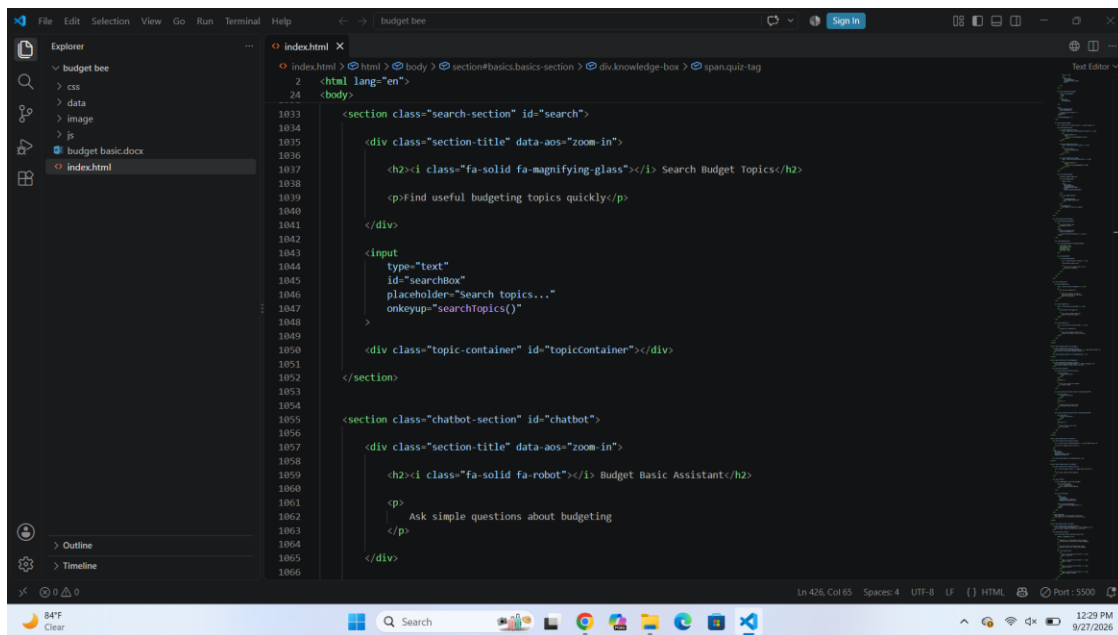
A screenshot of the Visual Studio Code editor interface. The Explorer panel on the left shows a file tree with 'index.html' selected. The main editor area displays the HTML code for 'index.html'. The code is structured as follows:

```
2 <html lang="en">
24 <body>
667 <section class="expense-section" id="expenses">
845 <div class="expense-table-container">
846 <div class="expense-table-header">
847 <div>
849 <span>YOUR EXPENSES</span>
850 <h3>Expense List</h3>
852 </div>
853 <button
854 class="clear-expenses-btn"
855 onclick="clearExpenses()"
856 >
857 <i class="fa-solid fa-trash-can"></i> Clear All
858 </button>
859 </div>
860 <div class="expense-table">
861 <div class="expense-table-row expense-heading">
862 <span>Expense</span>
863 <span>Category</span>
864 <span>Date</span>
865 <span>Amount</span>
866 <span>Action</span>
867 </div>
868 </div>
869 </div>
870 </div>
871 </div>
872 </div>
873 </div>
874 </div>
875 </div>
876 </div>
877 </div>
```

The status bar at the bottom indicates the file is 'index.html' at line 426, column 65, with 4 spaces, UTF-8 encoding, and LF line endings. The system tray shows a temperature of 84°F and the date 9/27/2026.

Code Reference 5

Code Reference 6



A screenshot of the Visual Studio Code editor interface. The Explorer panel on the left shows a file tree with 'index.html' selected. The main editor area displays the HTML code for 'index.html'. The code is structured as follows:

```
1033 <section class="search-section" id="search">
1034 <div class="section-title" data-aos="zoom-in">
1035 <h2><i class="fa-solid fa-magnifying-glass"></i> Search Budget Topics</h2>
1036 <p>Find useful budgeting topics quickly</p>
1037 </div>
1038 <input
1039 type="text"
1040 id="searchBox"
1041 placeholder="Search topics..."
1042 onkeyup="searchTopics()"
1043 >
1044 <div class="topic-container" id="topicContainer"></div>
1045 </section>
1046 <section class="chatbot-section" id="chatbot">
1047 <div class="section-title" data-aos="zoom-in">
1048 <h2><i class="fa-solid fa-robot"></i> Budget Basic Assistant</h2>
1049 <p>Ask simple questions about budgeting</p>
1050 </div>
1051 </div>
1052 </div>
1053 </div>
1054 </div>
1055 </div>
1056 </div>
1057 </div>
1058 </div>
1059 </div>
1060 </div>
1061 </div>
1062 </div>
1063 </div>
1064 </div>
1065 </div>
1066 </div>
```

The status bar at the bottom indicates the file is 'index.html' at line 426, column 65, with 4 spaces, UTF-8 encoding, and LF line endings. The system tray shows a temperature of 84°F and the date 9/27/2026.

Code Reference 6

Code Reference 7

```

1055 <section class="chatbot-section" id="chatbot">
1056   <div class="chatbot">
1057     <div id="chatMessages" class="chat-messages">
1058       <div class="bot-message">
1059         Hi! 🐝 I'm Budget Basic.
1060         Ask me something about budgeting.
1061       </div>
1062     </div>
1063     <div class="chat-input">
1064       <input
1065         type="text"
1066         id="chatInput"
1067         placeholder="Ask a question..."
1068         onkeypress="handlechat(event)"
1069       />
1070       <button onclick="sendMessage()"
1071         class="fa-solid fa-paper-plane"></i>
1072         Send
1073       </button>
1074     </div>
1075   </div>
1076   <p class="chat-note">
1077     Note: Budget Basic provides general educational
1078     information and is not professional financial advice.
1079   </p>
1080 </section>

```

Code Reference 7

Code Reference 8

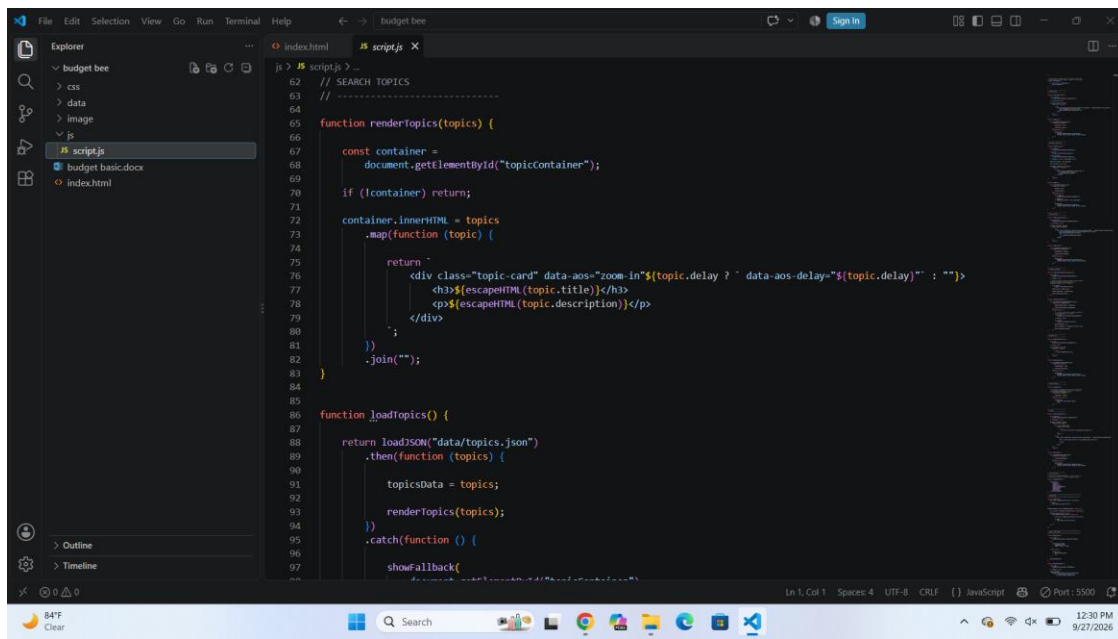
```

18 let topicsData = null;
19 let quizData = null;
20 let mistakesData = null;
21 let expenseCategoriesData = null;
22 let savingsData = null;
23 let chatbotData = null;
24
25 // Small helper: fetch a JSON file and return the parsed data.
26 // Throws on a bad response so the caller can show its fallback.
27 function loadJSON(url) {
28   return fetch(url).then(function (response) {
29     if (!response.ok) {
30       throw new Error("HTTP " + response.status);
31     }
32     return response.json();
33   });
34 }
35
36 // Shows a simple fallback message inside a section container
37 function showFallback(container, message) {
38   if (!container) return;
39   container.innerHTML =
40     '<p class="load-fallback">' + message + '</p>';
41 }
42
43 // Re-scan AQS after JSON content is rendered so the newly
44 // added data-aes elements animate instead of staying hidden
45 function refreshAQS() {

```

Code Reference 8

Code Reference 9



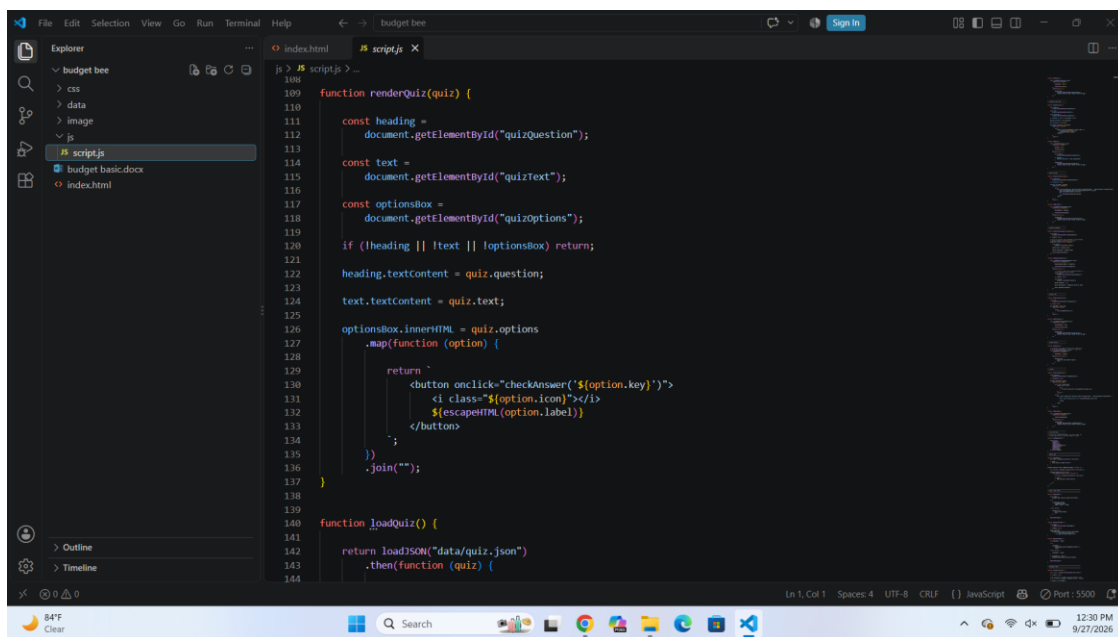
```

62 // SEARCH TOPICS
63 // -----
64
65 function renderTopics(topics) {
66
67   const container =
68     document.getElementById("topiccontainer");
69
70   if (!container) return;
71
72   container.innerHTML = topics
73     .map(function (topic) {
74
75       return `
76         <div class="topic-card" data-aos="zoom-in" ${topic.delay ? `data-aos-delay="${topic.delay}"` : ""}>
77           <h3>${escapeHTML(topic.title)}</h3>
78           <p>${escapeHTML(topic.description)}</p>
79         </div>
80       `;
81     })
82     .join("");
83
84 }
85
86 function loadTopics() {
87
88   return loadJSON("data/topics.json")
89     .then(function (topics) {
90
91       topicsData = topics;
92       renderTopics(topics);
93     })
94     .catch(function () {
95       showFallback(
96         "data/topics.json"
97       );
98     });
99 }

```

Code Reference 9

Code Reference 10



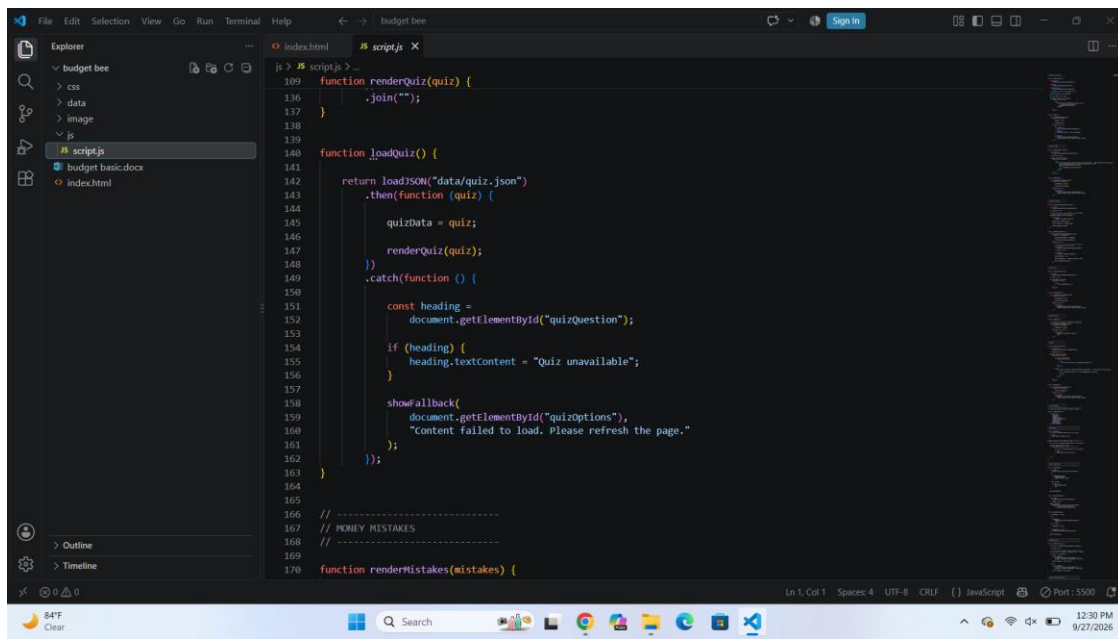
```

100
101 function renderQuiz(quiz) {
102
103   const heading =
104     document.getElementById("quizQuestion");
105
106   const text =
107     document.getElementById("quizText");
108
109   const optionsBox =
110     document.getElementById("quizOptions");
111
112   if (!heading || !text || !optionsBox) return;
113
114   heading.textContent = quiz.question;
115   text.textContent = quiz.text;
116
117   optionsBox.innerHTML = quiz.options
118     .map(function (option) {
119
120       return `
121         <button onclick="checkAnswer('${option.key}')">
122           <div class="${option.icon}"></div>
123           ${escapeHTML(option.label)}
124         </button>
125       `;
126     })
127     .join("");
128
129 }
130
131 function loadQuiz() {
132
133   return loadJSON("data/quiz.json")
134     .then(function (quiz) {
135
136     });
137 }

```

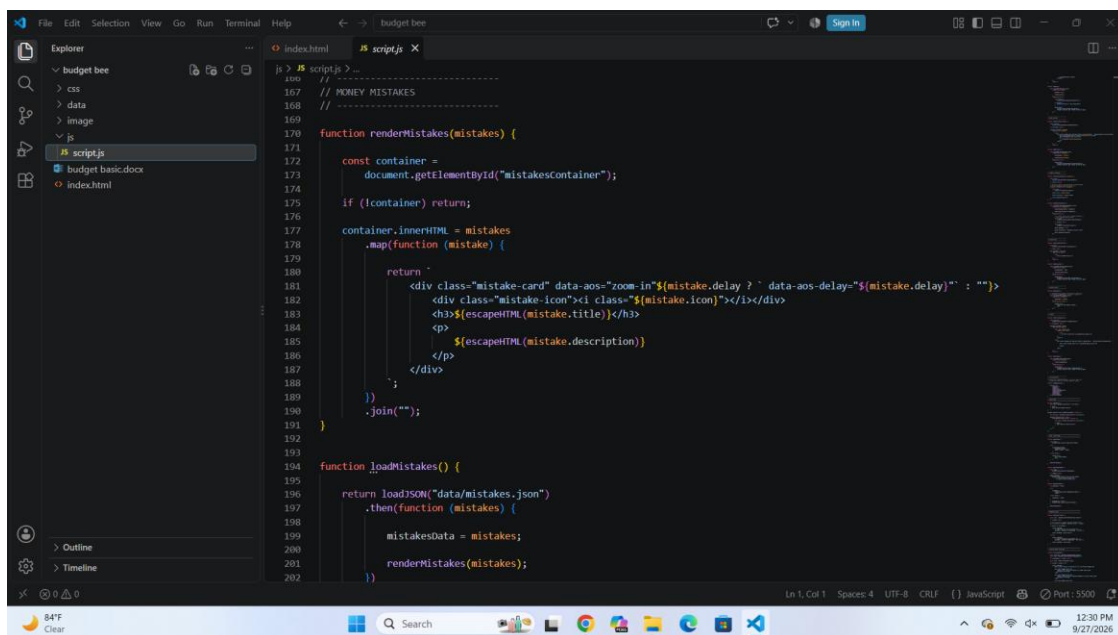
Code Reference 10

Code Reference 11



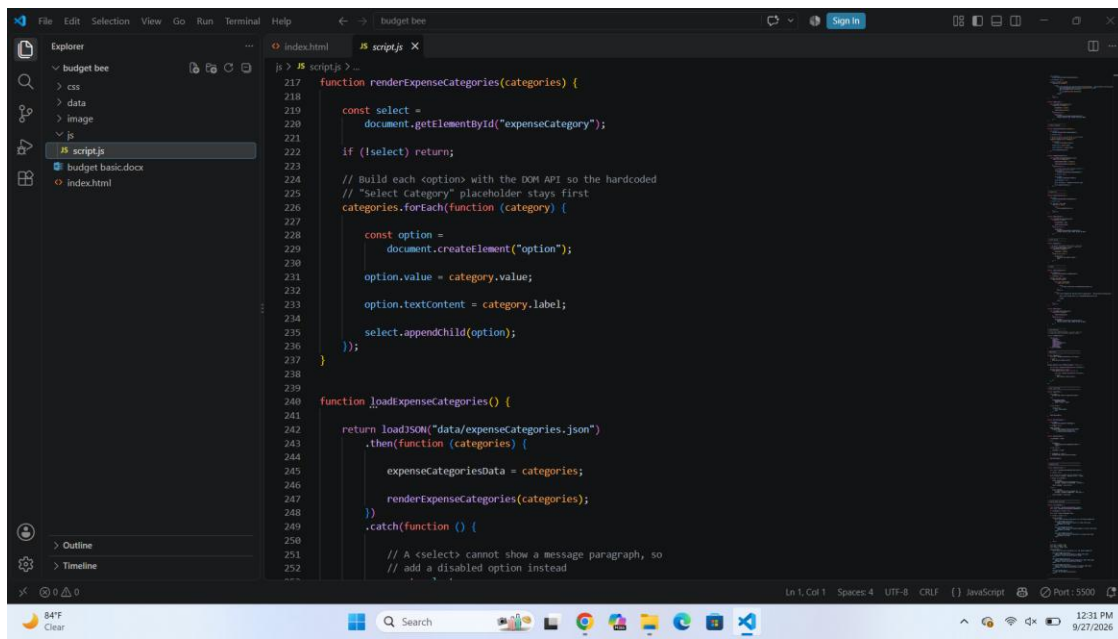
Code Reference 11

Code Reference 12



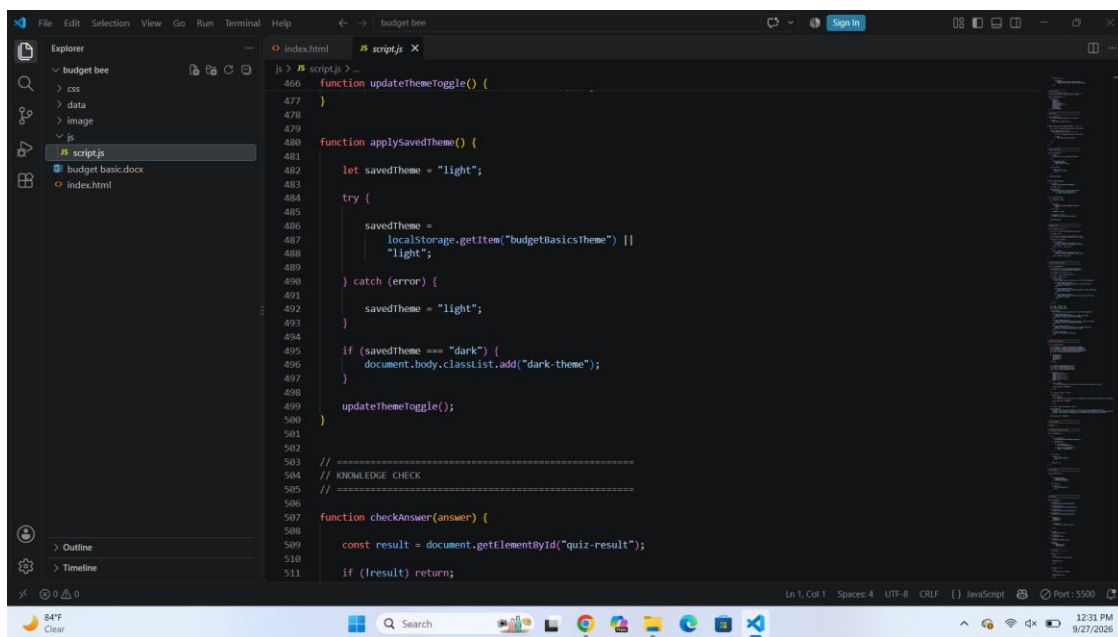
Code Reference 12

Code Reference 13



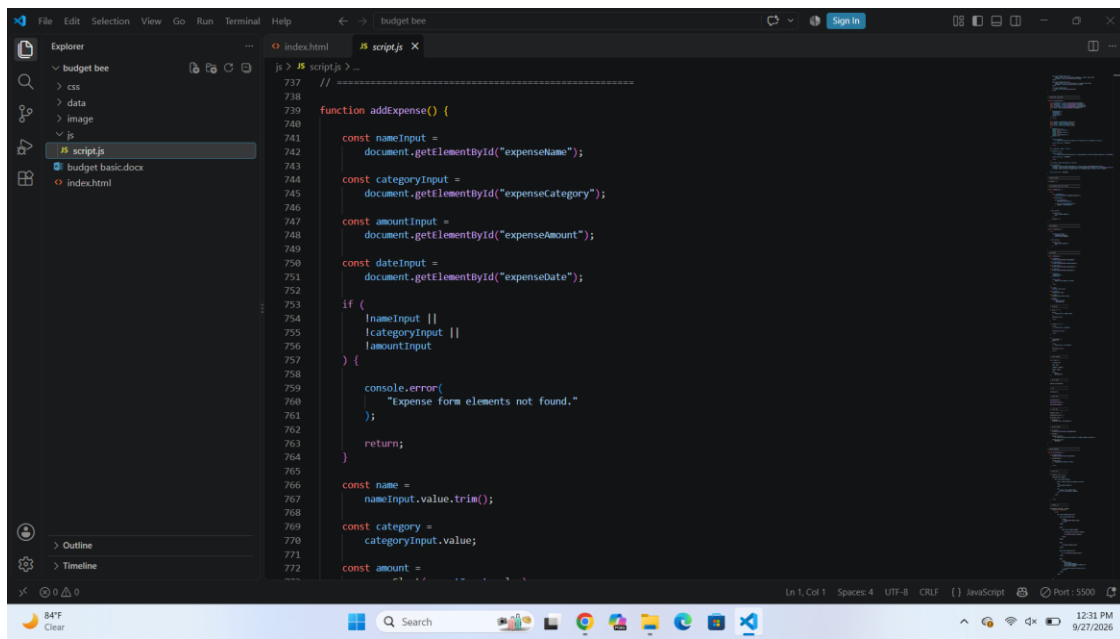
Code Reference 13

Code Reference 14



Code Reference 14

Code Reference 15



```
737 // =====
738
739 function addExpense() {
740
741     const nameInput =
742         document.getElementById("expenseName");
743
744     const categoryInput =
745         document.getElementById("expenseCategory");
746
747     const amountInput =
748         document.getElementById("expenseAmount");
749
750     const dateInput =
751         document.getElementById("expenseDate");
752
753     if (
754         !nameInput ||
755         !categoryInput ||
756         !amountInput
757     ) {
758
759         console.error(
760             "Expense form elements not found."
761         );
762
763         return;
764     }
765
766     const name =
767         nameInput.value.trim();
768
769     const category =
770         categoryInput.value;
771
772     const amount =
```

Code Reference 15

14. AI Tool Acknowledgement

In line with the SRS's AI usage guidelines, the following AI-assisted tools were used during development, with all output reviewed, tested, and meaningfully modified by the project team:

- Claude (Anthropic) — code review, refactoring guidance, CSS/JS debugging assistance, and this documentation.
- Font Awesome — icon library used for UI iconography.
- AOS (Animate On Scroll) — open-source scroll-animation library.

No full boilerplate or ready-made website template was used; the structure, styling system, and interactive logic reflect the team's own implementation and iterative refinement.

15. Conclusion

BudgetBasics successfully delivers on the SRS's functional and non-functional requirements: a responsive, accessible, single-page educational website that teaches budgeting fundamentals through interactive calculators, visual infographics, a searchable topic library, and a predefined-answer chatbot — all wrapped in a consistent, polished teal-and-gold visual identity, with all content sourced from external JSON files rather than hardcoded markup.

 *Learn • Plan • Save — BudgetBasics*